Would I Lie to You? Web App: Business Requirements Document

Summary

A browser game based on the TV panel show *Would I Lie to You?*. Two teams of 3 players take turns: a speaker reads a statement that is either one of their own true stories or an AI-generated lie, and the other team questions them for up to 10 minutes before guessing Truth or Lie. A correct guess wins the guessing team a point; a wrong guess gives the point to the speaker's team.

Objectives

1. Recreate the core bluffing loop of the show as a playable local web game.
2. Use AI in the product itself: generate believable lies that sit alongside players' real stories.
3. Demonstrate AI-assisted software development.
4. Keep the code in Python so it can be explained and extended by the developer.

Scope

In scope (v1)

- 3v3 only, on one shared device (hot-seat)
- Fixed 6 rounds per game, plus sudden death if scores are level
- True-story entry per player; lies from a free local AI model (Ollama) or the lie bank; test mode
- Round flow: privacy screen, spacebar start, 10-minute timer, Ready to Guess, T/L input, scoring
- Scoreboard and end-of-game winner screen
- Runs on localhost

Out of scope (v1)

- Online multiplayer across separate devices
- User accounts, saved history, leaderboards
- Video or voice features
- Public deployment (planned later, see Non-functional requirements)

Assumptions

- Players are in the same room; questioning happens out loud, not in the app.
- Ollama is installed on the laptop for AI lies; the lie bank needs nothing extra.
- One person per guessing team types the final T/L answer after the team confers.

Stakeholders and users

Who	Role	Needs
Abdul-Haque Mohammed	Product owner and developer	Clear, buildable requirements
Work experience team	Reviewers	Working demo of AI-assisted build and AI in the product
Speaker	Player reading the statement	Sees the statement privately; nobody else can
Guessing team	Players questioning the speaker	Timer, a way to guess early, simple T/L input

Game rules

A round is one statement read by one speaker. Teams alternate as speaker each round, and speakers rotate within a team so everyone reads.

The game is always 3v3 (6 players). Every game is 6 rounds, plus sudden death if scores are level.

Each player speaks once and each team speaks 3 times. Each player still enters 2 true stories, so the AI has a choice and the speaker's own pick is not predictable.

Round flow

1. App names the speaker and shows a privacy screen: the guessing team looks away.
2. Speaker reveals their statement privately. Behind the scenes the AI has picked either one of that speaker's unused true stories or a generated lie. Speaker hides it again.
3. Speaker presses Spacebar to start the round. The 10:00 timer starts.
4. Speaker reads the statement aloud; the guessing team questions them out loud.
5. Round moves to guessing when the timer hits 0:00 or the guessing team clicks **Ready to Guess**.
6. Guessing team types **T** (truth) or **L** (lie) and confirms.
7. App reveals TRUTH or LIE and shows which team won the point: the guessing team if correct, the speaker's team if wrong.
8. Next round, until all 6 are played. Highest score wins; if level, sudden death (FR22).

Functional requirements

ID	Requirement	Priority
FR1	Game is fixed at 3v3: two teams of 3 players	Must
FR2	User enters team names and player names	Must
FR3	Game runs for exactly 6 rounds (3 per team); no round selection; sudden death only if level (FR22)	Must
FR4	Each player privately enters exactly 2 true stories; the other team cannot see them	Must

FR5	The local AI model generates lies before the game starts, matching the style and length of the true stories	Must
FR6	If the AI call fails, the app falls back to a bank of pre-written lies so the game still runs. The bank holds at least 60 lies (6 per game x 10 games) so FR19 holds in offline mode	Must
FR7	Before each round the app randomly picks a true story or a lie for the speaker (independent coin flip each round, so the number of lies per game is not fixed); a true story is never reused	Must
FR8	Privacy screen before each round; statement shown only after the speaker chooses to reveal it, then hidden again	Must
FR9	Spacebar starts the round, only once the statement is hidden	Must
FR10	10:00 countdown timer, visible to everyone	Must
FR11	Ready to Guess button ends questioning early	Must
FR12	Guess input accepts only T or L (case-insensitive) and needs a confirm	Must
FR13	Scoring: correct guess = +1 to the guessing team; wrong guess = +1 to the speaker's team. No negative points	Must
FR14	Reveal screen shows the statement and whether it was TRUTH or LIE, and which team got the point	Must
FR15	Live scoreboard and round counter	Must
FR16	End screen with final scores, the winner, and Play Again	Must
FR17	AI-generated lies are family-friendly; offensive output is rejected and regenerated	Should
FR18	Sound effect when the timer ends	Could
FR19	No lie appears more than once within any 10 consecutive games, whether from the local AI or the lie bank; a repeat or near-duplicate is regenerated	Must
FR20	Beyond 10 games, lies still avoid repeating where possible: recent ones are passed to the AI as "do not reuse", and the lie bank is used up in full before any entry repeats	Should
FR21	Lie bank option on the setup screen: skips AI entirely; no API key or internet needed	Must
FR22	Sudden death if scores are level after 6 rounds: each team gets one more round (using players' unused true stories or new lies); the first pair of rounds that leaves one team ahead decides the winner, otherwise repeat	Must
FR23	Test mode toggle: shows TRUTH or LIE on screen for every statement during the round, for development and demos. Test games are labelled as such and do not count toward the used-lies history	Must
FR24	Setup screen offers a choice of lie source: local AI (a free model running in Ollama on the laptop, the default) or lie bank. If the chosen AI is unavailable, the game falls back to the lie bank and says so (CR-01, CR-02)	Must

Non-functional requirements

ID	Requirement
NFR1	Backend in Python (Flask or FastAPI). A small amount of JavaScript is unavoidable for the live timer and spacebar handling, since browsers do not run Python
NFR2	Runs with one command on localhost (e.g. `python app.py`)
NFR3	Deployable later to a host such as Render or Railway without code changes beyond config
NFR4	No secrets in the code; local settings live in a .env file that is never committed to Git
NFR5	All lies generated in a single batch before round 1, so no AI wait during play; target under 15 seconds
NFR6	Game state held in the server session. The only thing kept after a game is the used-lies history (a local JSON or SQLite file); players' true stories and names are never stored
NFR7	Usable on a laptop screen in Chrome, Safari and Edge
NFR8	Code split into clear modules (game logic, AI service, routes) with unit tests for scoring and round-count rules

Risks

Risk	Impact	Mitigation
One shared screen means the guessing team could glance at the statement	Round is spoiled	Privacy screen + reveal-then-hide (FR8); later, speaker views the statement on their phone
AI lies are obviously fake or wildly different in tone from true stories	Game is too easy	Prompt includes the player's true stories as style examples
Local AI not installed, not running or slow	Lies come only from the lie bank	Automatic fallback to the lie bank with a message (FR6, FR24)

Open questions

- Is a phone view for the speaker wanted in v2?

Success criteria

- A full 3v3 game can be played end to end on localhost with no errors.
- Scoring, sudden death, no-repeat lie rules and T/L validation pass unit tests.
- In playtesting, guessers cannot reliably tell AI lies from true stories (target: under 70% correct).

Version history

Version	Date	Change

1.0	12 Nov 2026	First signed-off version
1.1	12 Nov 2026	CR-01 approved: FR24 local AI lie source added; FR21 reworded
1.2	12 Nov 2026	CR-02 (awaiting client approval): Claude removed; lies come from the local AI or the lie bank; FR5, FR19, FR20, FR24, NFR4, assumptions and risks updated